Speech & Speaker Recognition System

Farhan Farooq Famim
ID: 2222100042
Email: farhan.famim@northsouth.edu

Mehbur Rahman
ID: 2121673042
Email: mehbur.rahman@northsouth.edu

Nahian Islam Inan
ID: 2112259642
Email: nahian.inan@northsouth.edu

Asmaul Islam Tisha
ID: 2021934642
Email: asmaul.tisha@northsouth.edu

Group - 06 | Course code: CSE445 | Section: 06

1. Abstract

This project presents the design and implementation of an intelligent audio processing system that integrates two core capabilities: automatic speech recognition (ASR), and speaker identification. The system leverages OpenAI's Whisper model to transcribe spoken audio into text, a Support Vector Machine (SVM) classifier trained on 39-dimensional Mel-Frequency Cepstral Coefficient (MFCC) feature vectors to identify the speaker from their voice, and a custom Convolutional Neural Network (CNN) trained on log-normalized spectrograms to classify audio clips into one of ten predefined voice commands. The combined pipeline accepts raw audio input and produces two parallel outputs — a text transcription, and a speaker identity — making it suitable for intelligent assistant and home automation scenarios. The system is designed to run entirely on CPU, making it accessible without specialized hardware. Evaluation is performed using Word Error Rate (WER) for transcription accuracy and classification accuracy for both the speaker and command recognition components.

2. Introduction

Voice-based interaction has become one of the most natural and convenient interfaces between humans and machines. From virtual assistants like Siri and Alexa to smart home systems and accessibility tools, the ability to understand not just *what* is being said, but *who* is saying it and *what action* is intended, represents a critical step toward truly intelligent systems. This project addresses all three of these dimensions in a unified pipeline.

The system is divided into two main modules developed collaboratively. The first module handles speech-to-text transcription and speaker recognition. Transcription is performed using Whisper, a state-of-the-art ASR model developed by OpenAI, which is capable of handling diverse audio conditions and multiple languages. Speaker recognition is implemented using a classical machine learning approach — an SVM classifier trained on MFCC-based acoustic features extracted from each speaker's voice recordings. The second module introduces a custom-trained deep learning model for voice command detection. Rather than relying on general-purpose transcription to infer intent, this module uses a CNN that directly processes the spectral representation of audio to classify it into one of ten specific commands, such as "Turn on the light" or "Play the music."

The two modules are integrated through a shared manifest-based pipeline, where a CSV file describes the dataset — including audio paths, ground-truth transcripts, speaker IDs, languages, and command labels — and a unified evaluation script processes every entry, producing results for all three tasks simultaneously.

3. Objectives

The project was undertaken with the following specific objectives:

- **Speech-to-Text Transcription:** Implement an accurate automatic speech recognition system capable of converting spoken audio into text, and evaluate its performance using Word Error Rate (WER) against ground-truth transcripts.
- **Speaker Identification:** Build a system that can identify which speaker is talking from a pool of known speakers, using acoustic features extracted from their voice, without relying on any text content of the speech.
- **Voice Command Detection:** Train a custom model to recognize ten predefined voice commands directly from audio, enabling intent detection that is faster and more robust than relying solely on transcription.
- **System Integration:** Combine all three components into a single, coherent pipeline driven by a structured manifest file, so that a single run of the system produces transcription, speaker identity, and command classification for every audio file in the dataset.
- **CPU Accessibility:** Ensure that the entire system — including model training and inference — runs efficiently on standard hardware without requiring a dedicated GPU, making it broadly accessible.
- **Modular Design:** Keep each component (transcription, speaker recognition, command detection) independently replaceable and testable, so future improvements to one part do not require rewriting the others.

4. Motivation

The motivation behind this project stems from both practical and academic perspectives. Voice-based interfaces are increasingly becoming the primary mode of interaction in smart environments — homes, offices, and assistive technologies for individuals with physical disabilities. However, most readily available solutions are either cloud-dependent, requiring constant internet connectivity and raising privacy concerns, or too computationally heavy to run on everyday consumer hardware. This project was motivated by the desire to build a system that is lightweight, offline-capable, and modular enough to be extended or repurposed.

From an academic standpoint, the project offered the opportunity to work across two distinct paradigms of machine learning — classical learning (SVM with handcrafted features) and deep learning (CNN with learned representations) — and to understand the trade-offs between them in a real-world audio processing context. Speaker recognition using MFCCs and SVMs represents a well-established signal processing approach, while the CNN-based command classifier represents a more modern, data-driven methodology. Implementing both within the same system allowed for a direct, practical appreciation of where each approach excels.

Additionally, the group's limited dataset — 5 speakers with 5 recordings each, and a small set of command audio samples — made the project a realistic exercise in building systems under constrained conditions, which is far more representative of real-world scenarios than

working with large, curated benchmarks. This constraint pushed the team to think carefully about feature engineering, data organization, and architectural choices rather than relying on scale alone.

5. Novelty

While each individual component of this system — Whisper-based transcription, MFCC-based speaker recognition, and CNN-based command classification — exists independently in prior work, the novelty of this project lies in the following aspects:

- **Three-task unified pipeline:** Most existing student or small-scale projects address speech transcription, speaker recognition, or command detection in isolation. This system combines all three into a single, manifest-driven pipeline that produces all outputs simultaneously for each audio file, without any manual intermediate steps.
- **Manifest-based evaluation architecture:** The use of a structured CSV manifest file (containing audio paths, transcripts, speaker IDs, languages, command labels, and dataset splits) is a design choice borrowed from professional ASR system design. It makes the dataset fully declarative — adding new speakers, commands, or audio files requires no code changes, only updates to the manifest.
- **Dual-paradigm approach to audio classification:** The system deliberately uses two different machine learning paradigms side by side — a classical SVM with handcrafted 39-dimensional MFCC feature vectors for speaker recognition, and a deep CNN operating on log-normalized spectrograms for command detection. This is not accidental; it reflects a conscious design decision to apply the most appropriate tool to each sub-problem given the available data.
- **Resource-conscious design:** The entire system is designed and validated to run on CPU-only hardware with an integrated GPU. This is a non-trivial constraint that influenced several design decisions, including the choice of Whisper's base model, the use of fp16=False during inference in transcriber.py, and the preference for SVM over more data-hungry deep learning models for speaker recognition.
- **Graceful degradation:** The system is built to handle missing data gracefully — missing transcripts skip WER calculation but still produce transcription and speaker output, missing audio files are logged without crashing the pipeline, and a missing speaker model disables that component with a warning rather than halting the program entirely.

6. Methodology

The system is built as three interconnected modules. Each is described below in detail.

6.1 Speech Transcription (Whisper)

Audio transcription is handled by OpenAI's Whisper, a transformer-based sequence-to-sequence ASR model trained on 680,000 hours of multilingual audio. The base model variant is used, which balances accuracy and speed for CPU inference. The transcriber.py module manages model loading with a thread-safe cache so the model is loaded only once per session regardless of how many files are processed. For .wav files, audio is loaded and resampled to 16 kHz using a custom load_wav_audio() function that avoids external dependencies. For all other formats (.mp3, .m4a, etc.), the file path is passed directly to Whisper, which uses ffmpeg internally.

Transcription accuracy is evaluated using Word Error Rate (WER), computed after normalizing both the ground-truth and predicted text to lowercase with punctuation removed.

6.2 Speaker Recognition (MFCC + SVM)

Speaker recognition follows a two-stage process: feature extraction and classification.

Feature Extraction: For each audio file, feature_extraction.py loads the audio at 16 kHz using librosa and computes 13 Mel-Frequency Cepstral Coefficients (MFCCs). From these, three statistics are derived and concatenated into a single 39-dimensional feature vector:

- **MFCC Mean** (13 values) — captures the average tonal and timbral characteristics of the speaker's voice
- **MFCC Standard Deviation** (13 values) — captures variability in the voice, reflecting speaking rhythm and style
- **Delta MFCC Mean** (13 values) — captures the rate of change of MFCCs over time, reflecting voice transitions between sounds

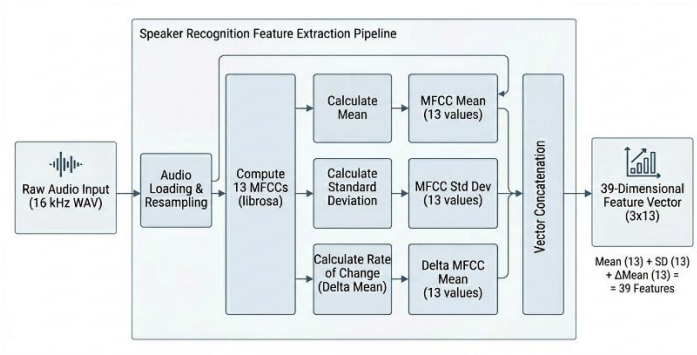

Figure 1: 39-dimensional feature extraction SVM pipeline.

Classification: The feature vectors from all training audio files are used to train a linear kernel SVM using scikit-learn. The dataset is split 70/30 into training and test sets. The trained model is serialized to disk using joblib as speaker_model.pkl, and raw features are cached as .npy files to avoid recomputation. At inference time, the model is loaded once and applied to each audio file in the pipeline to produce a speaker label.

MFCC Feature Extraction:

The Mel filterbank conversion from frequency to Mel scale:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

The 39-Feature Vector Construction:

$$\mathbf{x} = [\mu_{MFCC} \;\|\; \sigma_{MFCC} \;\|\; \mu_{\Delta MFCC}]$$

Where:

— $\mu_{MFCC} \in \mathbb{R}^{13}$ — mean of MFCC coefficients across time

— $\sigma_{MFCC} \in \mathbb{R}^{13}$ — standard deviation of MFCC coefficients across time

— $\mu_{\Delta MFCC} \in \mathbb{R}^{13}$ — mean of delta MFCCs across time

— $\|$ denotes vector concatenation

— $\mathbf{x} \in \mathbb{R}^{39}$ — the final feature vector passed to the SVM

SVM Decision Function (Linear Kernel):

Since you're using a linear kernel (SVC(kernel='linear')), the decision function is:

$$\boldsymbol{f}(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + \boldsymbol{b}$$

Where $\mathbf{w}$ is the weight vector and b is the bias term. The predicted speaker label is:

$$\hat{y} = \arg\max_{k} \; \boldsymbol{f}_k(\mathbf{x})$$

Where k indexes each speaker class.

SVM Optimization Objective:

The SVM learns w and b by solving:

$$\min_{\mathbf{w},b} \quad \frac{1}{2} \| \mathbf{w} \|^2 \text{ subject to} y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \quad \forall i$$

This is the hard-margin SVM objective. Include this to show the mathematical basis of why the SVM finds the maximum margin boundary between speaker classes.

6.3 Voice Command Detection (CNN)

The command detection module, implemented in command_model.py and trained via train_command_model.py, classifies audio into one of ten predefined commands:

Feature Representation: Each audio clip is loaded, resampled to 16 kHz, and padded or trimmed to exactly 3 seconds. A Short-Time Fourier Transform (STFT) is applied to produce a spectrogram, which is then log-compressed with log1p and z-score normalized. This produces a 2D time-frequency representation that is passed to the CNN as a single-channel image.

Model Architecture (CommandCNN): The CNN consists of three convolutional blocks, each with a Conv2D layer, Batch Normalization, ReLU activation, and Max Pooling, followed by an Adaptive Average Pooling layer that normalizes spatial dimensions to 8×8 regardless of input length. A fully connected classifier with Dropout then maps the 4096-dimensional flattened feature to the 10 command classes.

Training: The model is trained using the Adam optimizer with cross-entropy loss. Training, validation, and test splits are defined in the manifest CSV. The best checkpoint (by validation accuracy) is saved to artifacts/command_model/best_command_model.pt and reloaded for final test evaluation. At inference time, the model outputs a confidence score via softmax alongside the predicted command ID and its human-readable text.

6.4 Real-Time Audio Input

In addition to processing pre-recorded audio files, the system supports real-time microphone input using the pyaudio library. This transforms the system from a batch evaluation tool into an interactive voice interface. When real-time mode is activated, the system continuously listens through the microphone, captures audio in a rolling buffer, and applies voice activity detection to identify when a user has started and finished speaking. Once a complete utterance is detected, the captured audio is passed through the same pipeline as pre-recorded files — Whisper transcribes it, the SVM identifies the speaker, and the CNN classifies the command — all within seconds of the user finishing their sentence. The audio is sampled at 16 kHz to match the input requirements of all three models, ensuring no resampling overhead during live inference. This real-time capability represents the most user-facing aspect of the system and demonstrates that the pipeline is not only accurate in offline evaluation but also responsive enough for practical interactive use on CPU-only hardware.

Command ID	Command Text
light_on	Turn on the light
light_off	Turn off the light
door_open	Open the door
door_close	Close the door
music_play	Play the music
.....	.....

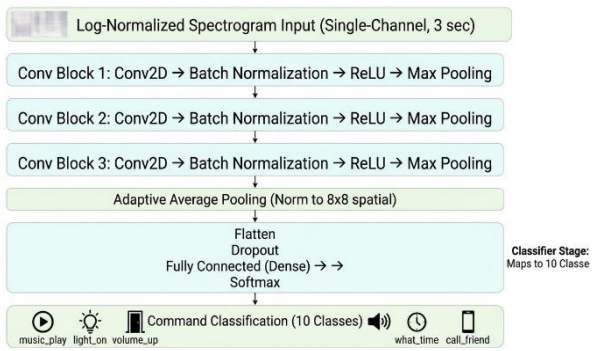

Figure 2: CommandCNN model architecture.

6.5 Integrated Pipeline

The three modules are unified through main.py and transcriber.py. A single CSV manifest file

describes the entire dataset. Running main.py with optional command-line arguments for model size, manifest path, and dataset split triggers the full pipeline — each audio file is transcribed by Whisper, identified by the SVM, and classified by the CNN, with all results printed to the console in a structured format.

7. System Architecture

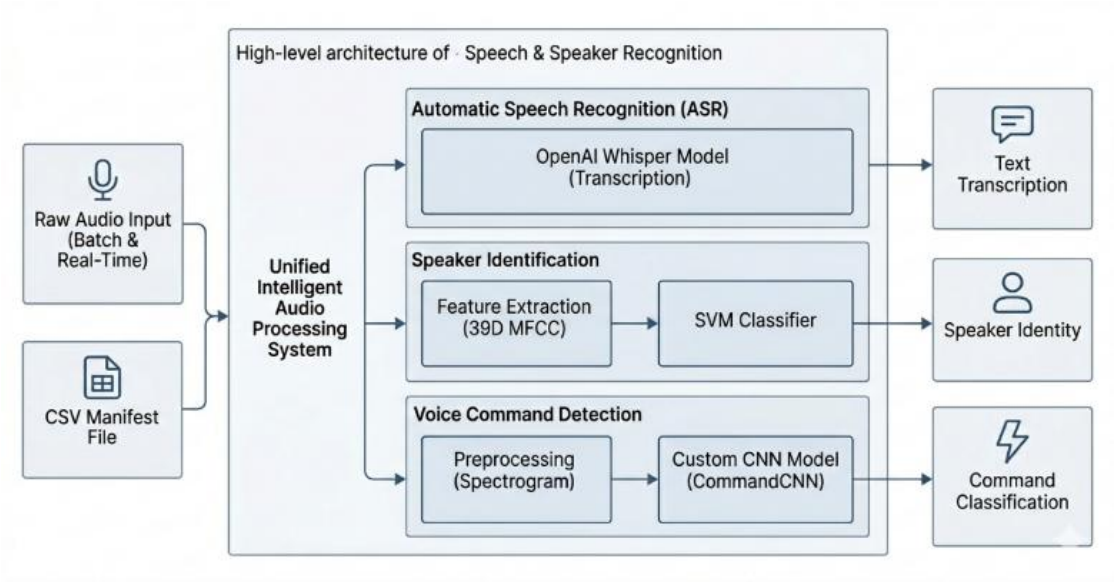

Figure 3: *System Architecture of the Speech and Speaker Recognition system*

8. Challenges Faced

Building this system from scratch under real-world constraints presented several notable challenges across both modules of the project.

Limited Dataset Size

The most significant challenge throughout the project was the scarcity of training data. The speaker recognition module was trained on only 25 audio files — 5 recordings per speaker across 5 speakers. This is far below what a robust speaker recognition system typically requires. With so few samples, the SVM struggles to generalize to new recordings of the same speaker, particularly if the recording conditions differ even slightly from the training data. The same constraint applied to the command detection module, where the number of training samples per command class directly impacts the CNN's ability to distinguish between acoustically similar commands such as "Turn on the light" and "Turn off the light."

Recording Condition Sensitivity

MFCC-based features are highly sensitive to recording conditions. During testing, it was observed that even the same speaker recorded on a different device or in a different room produced MFCC values that differed noticeably from the training samples. This caused the SVM to misidentify speakers even when the voice was clearly recognizable to a human listener. This is an inherent limitation of shallow acoustic features combined with a small training set and is not easily resolved without either collecting more

diverse training data or switching to a pretrained speaker embedding model.

Audio Format Compatibility

Supporting multiple audio formats introduced unexpected complexity. The librosa library requires ffmpeg to be installed on the system to handle formats beyond WAV. This is an external system dependency that is not automatically installed with Python packages, and its absence causes silent failures — *extract_features()* returns None and the speaker is reported as unknown with no clear error message to the user. Managing this gracefully required adding fallback messages and defensive checks throughout the code.

Manifest Design and Data Organization

Designing the CSV manifest schema to serve all three tasks simultaneously — transcription, speaker recognition, and command classification — required careful planning. Each row needed to carry enough metadata (audio path, transcript, speaker ID, language, command ID, split) without becoming redundant or inconsistent. Keeping the manifest synchronized with the actual files on disk, especially as new recordings were added or folders were renamed, was an ongoing organizational challenge during development.

Integration Between Two Independent Modules

Since the two parts of the project were developed independently by different team members, integrating them into a single cohesive pipeline required reconciling different assumptions about file paths, audio loading, and output formats. For example, the speaker recognition module used relative paths anchored to the dataset/ folder, while the command detection module used paths relative to the manifest file's location. Aligning these conventions without breaking either module required careful refactoring of path handling throughout the codebase.

9. Future Plans

The current system, while functional, represents a foundation that can be meaningfully extended in several directions. The following improvements are planned or recommended for future iterations of the project.

Adopting a Pretrained Speaker Embedding Model

The most impactful near-term improvement to speaker recognition would be replacing the MFCC + SVM pipeline with a pretrained speaker embedding model such as resemblyzer or NVIDIA's SpeechBrain. These models were trained on thousands of speakers and encode voice characteristics into rich 256-dimensional embeddings. Rather than training a classifier from scratch, the system would simply compare the embedding of a new audio clip against stored reference embeddings using cosine similarity. This approach requires very few reference recordings per speaker — as few as one or two clips — making it far more suitable for small datasets like the one used in this project.

Confidence Thresholding and Unknown Speaker Handling

Both the SVM and the CNN currently always output a prediction regardless of how uncertain they are. In practice, this means a completely unfamiliar speaker will still be assigned one of the known speaker names, and an unrecognized audio clip will still be mapped to one of the ten commands. Adding confidence thresholding — where predictions below a set confidence score are reported as "Unknown" rather than forced into a known class — would make the system significantly more honest and reliable in real-world use.

Expanding the Command Set

The current system supports exactly ten commands, all hardcoded in command_model.py. Future work could make this extensible — allowing users to define new commands via a configuration file, record their own samples, and

retrain the model without modifying any source code. Pairing this with data augmentation would make it feasible to add new commands with as few as 10–15 recordings.

10. Conclusion

This project successfully demonstrates the design and implementation of a multi-task intelligent audio processing system that combines automatic speech recognition, speaker identification, and voice command detection into a unified pipeline. By integrating OpenAI's Whisper model for transcription, a classical SVM classifier trained on 39-dimensional MFCC feature vectors for speaker recognition, and a custom CNN operating on log-normalized spectrograms for command classification, the system is capable of processing a raw audio file and simultaneously answering three distinct questions — what was said, who said it, and what action was intended.

A key strength of the project is its thoughtful architecture. The manifest-based pipeline makes the system fully data-driven, meaning new speakers, commands, or audio files can be added without modifying any source code. The modular design ensures that each component can be improved, swapped, or tested independently. Graceful degradation throughout the codebase ensures the system continues to produce partial results even when data is incomplete, rather than failing entirely.

The project also provided honest exposure to the realities of building machine learning systems under practical constraints. The limited dataset was the most persistent challenge, and it revealed a fundamental tension between the simplicity of classical approaches like SVM and the data requirements of deep learning. The speaker recognition module, in particular, highlighted how sensitive acoustic features are to recording conditions, and motivated a clear path toward more robust pretrained embedding approaches in future work.

Overall, this project served as a comprehensive exercise in applied machine learning, audio signal processing, software integration, and system design. It bridges classical and modern approaches to audio understanding, runs entirely on consumer hardware, and produces a working system that could realistically serve as the backbone of a lightweight, privacy-preserving voice assistant. The groundwork laid here provides a clear and well-structured foundation for the future improvements outlined in the previous section.

---

11. References

Models and Libraries

- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision.* OpenAI. https://openai.com/research/whisper
- McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). *librosa: Audio and Music Signal Analysis in Python.* Proceedings of the 14th Python in Science Conference. https://librosa.org

Tools and Frameworks

- Morris, A. C., Maier, V., & Green, P. (2004). From WER and RIL to MER and WIL: Improved Evaluation Measures for Connected Speech Recognition. Interspeech 2004. (basis for the jiwer library)
- NumPy: Harris, C. R., et al. (2020). Array Programming with NumPy. Nature, 585, 357–362. https://numpy.org
- Joblib Development Team. (2020). Joblib: running Python functions as pipeline jobs. https://joblib.readthedocs.io